

VIRTUAL TECHNICAL CONFERENCE PLATFORM

SRN: PES1UG24CS551

Date: 16th Sep 2026

NAME: Abhay Dubey H

LAB-3: Component Modelling & Architectural Pattern Selection

Justification for the Architecture Used

Architecture Selection:

I chose Microservices as the Architecture for my Virtual Technical Conference Platform.

Reasons are as follows:

1. Independent Scaling for Peak Loads

The platform has uneven load patterns across its functions: Schedule Generator runs an Optimizer Algorithm for conference setup (i.e., FR-003 from my Requirements Table), while Live Q&A service should handle bursty, high-frequency upvote when sessions is active (FR-004). A Microservice Architecture allows for service to scale independently based on its own load profile. This directly supports NFR-001, which requires schedule generator to resolve room & speaker in 10 seconds or less, even under 50-concurrent-tracks benchmark.

2. Fault Isolation Across Independent Workflows

This platform groups many largely independent workflows – double-blind paper review (FR-001, FR-005), multi-track scheduling (FR-003), and live Q&A (FR-004) – each taken care of by a different actor (Author, Reviewer, Track Chair and Attendee). Microservices isolate the features within that boundary: if one of the services experiences a spike-induced fault during live sessions, the other features & services remain unaffected. A layered or client-server Architecture would cause a single fault to affect multiple services.

Security Advantage

Each microservice owns a narrow, well-defined boundary, which lets sensitive concerns be enforced close to the data protected. The Paper Submission & Review Service is the only path to manuscript content, so author-identity stripping for double-blind review (FR-001) and AES-256 at-rest encryption of stored papers and reviewer feedback (NFR-002) can be enforced consistently inside that one service and the Conference Database it depends on, rather than being re-implemented (and potentially missed) across a monolith's shared codebase. Routing all external traffic through a single API Gateway further centralizes authentication and access control before requests ever reach internal services.

Performance Benefit

Because the Schedule Generator Service is deployed and scaled independently, it can be given dedicated compute resources for its optimization algorithm without contention from unrelated traffic, such as concurrent Live Q&A upvotes. This isolation is what makes it feasible to reliably meet the sub-10-second scheduling benchmark defined in NFR-001, even as attendee-facing traffic on other services grows.